

L11 Limbajul SQL

Vederi (views)

1. Obiective

- Formarea și dezvoltarea abilităților de interogare a bazelor de date.
- Probleme rezolvate

2. Considerații teoretice

O **vedere** este o descriere parțială sau particulară a datelor din unul sau mai multe tabele ale bazei de date. O vedere crează un tabel virtual al cărui conținut este definit printr-o interogare.

Vederile construiesc tabele logice dinamice cu date care provin din tabelele aflate în baza de date. Spre deosebire de tabel, vederea *nu stochează date* și nici *nu are alocat un spațiu de stocare*; vederea doar *extrage sau derivă datele din tabelele* la care aceasta se referă. Aceste tabele poartă denumirea de *tabele de bază* ale vederii. Tabele de bază ale vederii pot fi tabele fizice sau pot fi ele însele vederi.

O vedere este o definiție a unei fraze SELECT memorată în baza de date. În principiu vederile se comportă și pot fi folosite la fel ca orice tabel, dar în cazul operațiilor de ștergere, inserare și actualizare sunt diverse restricții care trebuie să fie respectate.

Avantajul vederilor:

- Vederile simplifică și specializează viziunea utilizatorilor asupra datelor. Ele oferă grupurilor de utilizatori un model al datelor mai apropiat de nevoile lor.
- Vederile oferă un mecanism suplimentar de securitate. Grupuri de utilizatori pot avea asupra vederilor drepturi specifice diferite de drepturile acestora la nivelul tabelelor de bază;
- Prin proiectarea adecvată a vederilor, interogările și operațiile de manipulare a datelor pot fi rezolvate mai simplu. De exemplu, este posibilă definirea unei vederi care realizează un join între o vedere care include clauza GROUP BY și un alt tabel;
- Vederile mențin calculele complicate asupra datelor. Interogarea care definește vederea poate efectua calcule complicate asupra datelor din tabele; prin menținerea acestei interogări ca o vedere, calculele sunt efectuate de fiecare dată când se face referire la vedere.

Crearea unei vederi

Sintaxa pentru crearea unei vederi este următoarea:

```
CREATE VIEW denumireVedere[(denumireColoana [,denumireColoana]...)]  
    [WITH ENCRYPTION]  
AS fraza_SELECT  
    [WITH CHECK OPTION]
```

unde:

- `denumireColoana` specifică denumirile expresiilor selectate de interogarea asociată vederii. Numărul coloanelor vederii trebuie să fie același cu numărul de expresii selectate de către interogarea vederii. Dacă lista denumirilor coloanelor vederii este omisă, SQL va folosi denumirile coloanelor atribuite prin fraza SELECT;

- **WITH ENCRYPTION** determină stocarea interogării în formă criptată și prin urmare ea nu mai poate fi afișată;
- **AS** indică interogarea asociată vederii. Aceasta poate fi orice frază **SELECT** care nu conține clauzele **ORDER BY** și **INTO**;
- **WITH CHECK OPTION** este o constrângere care arată că toate actualizările efectuate prin intermediul vederii vor afecta tabelele de bază numai dacă actualizările respective vor avea ca rezultat doar rânduri care pot fi vizualizate prin intermediul vederii.

Observație. Este ușor să confundăm o vedere cu un tabel. Odată definită o vedere, datele pot fi accesate prin intermediul ei ca și cum ar fi un tabel. Din această cauză este bine să folosim convenții de nume: putem prefixa tabelele cu “t” și vederile cu “v”.

Exemple:

```
CREATE VIEW vSalariatiProductie (CodAngajat, Nume, Prenume,
CNP)
AS SELECT CodAngajat, Nume, Prenume, CNP
FROM tAngajati
WHERE CodDepartament = 'Productie'
```

În loc să obligăm utilizatorii să formuleze interogări complicate, putem defini astfel de interogări sub formă de vedere, permițându-le utilizatorilor să le acceseze:

```
Create VIEW vFacturiClienti
AS SELECT A.NrFact, A.DataFact, A.CodClient, C.NumeClient AS
Client, C.CodJudet, C.Localitate, B.CodAngajat, B.Nume+
'+B.Prenume AS Angajat, D.CodProd, E.DenProd, E.UM, D.Cantitate,
E.Pret, Cantitate*Pret AS Valoare
FROM
    tFacturi AS A inner JOIN tAngajati AS B ON
A.CodAngajat=B.CodAngajat
    INNER JOIN tClienti AS C ON A.CodClient=C.CodClient
    INNER JOIN tDetaliiFact AS D on A.NrFact=D.NrFact
    INNER JOIN tProduse AS E ON D.CodProd=E.CodProd
```

Putem utiliza acum vederea `vFacturiClienti` ca și cum ar fi un tabel:

```
SELECT * FROM vFacturiClienti WHERE MONTH(data)=3
```

Ca regulă, în **CREATE VIEW** nu este permisă folosirea clauzei **ORDER BY**. Există însă o posibilitate de a ocoli această restricție prin utilizarea extensiei **TOP 100 PERCENT**. **TOP n PERCENT** permite utilizarea clauzei **ORDER BY** în interiorul vederii dându-ne astfel voie să solicităm întregul tabel logic pentru $n=100$.

```
CREATE VIEW vSalariatiProductie2 (cod, nume, prenume,
salariu)
AS SELECT top 100 percent CodAngajat, nume, prenume, cnp
FROM tAngajati
WHERE CodDepartament = 'Productie'
ORDER BY Nume, Prenume
```

Utilizarea vederilor în subinterogări

1) Să se determine clienții pentru care suma valorică a facturilor ($\text{sum}(\text{cantitate} \cdot \text{pret})$) este mai mare decât valoarea medie facturată la nivel de client.

Vom crea o vedere care ne furnizează valoarea totală a facturilor la nivel de CodClient

```
CREATE VIEW vValoriFacturateClienti
AS SELECT A.CodClient,sum(Cantitate*Pret) AS Valoare
FROM
    tFacturi AS A inner JOIN tDetaliiFact AS B on
    A.NrFact=B.NrFact
    INNER JOIN tProduce AS C ON B.CodProd=C.CodProd
GROUP BY A.CodClient
```

Acum selectăm clienții solicitați în enunț

```
SELECT A.CodClient,B.NumeClient,B.CodJudet,B.Localitate,
    Valoare as [Valoare facturata]
FROM vValoriFacturateClienti AS A
    INNER JOIN tClienti AS B ON A.CodClient=B.CodClient
WHERE Valoare >(select AVG(Valoare) FROM
vValoriFacturateClienti)
```

Operații DML asupra vederilor

În momentul în care în tabelele de bază sunt adăugate noi date sau sunt actualizate sau șterse cele existente, aceste modificări se reflectă corespunzător în vederile bazate pe aceste tabele. Acest lucru este adevărat și reciproc, cu singura mențiune că există anumite restricții la inserarea, actualizarea sau ștergerea datelor dintr-o vedere. Aceste restricții sunt redactate pe scurt în continuare:

- Nu pot fi inserate, șterse sau actualizate datele din vederi care conțin una dintre următoarele:
 - operatorul DISTINCT (pentru eliminarea duplicatelor);
 - cuvântul cheie TOP
 - clauzele GROUP BY
 - funcțiile de grup (COUNT, SUM, MAX, MIN, AVG etc);
 - operatorul UNION
- Nu pot fi inserate noi rânduri dacă vederea conține coloane care rezultă prin calcul.
- Coloanele referite într-o frază UPDATE sau INSERT trebuie să aparțină unei singure tabele din cele referite în definiția vederii, astfel încât să se poată determina fără ambiguități tabela de bază la care se referă operația.
- Nu pot fi inserate sau actualizate date care ar încălca constrângerile din tabelele de bază. De exemplu, dacă în tabela *tAngajati* coloana *salariu* este definită ca NOT NULL, atunci în orice vedere bazată pe acest tabel care nu conține coloana *salariu* nu va fi posibilă inserarea de date deoarece aceasta ar duce la încălcarea constrângerii NOT NULL. Astfel următorul insert va furniza eroare

```
INSERT INTO vSalariatiiProductie(CodAngajat,Nume,Prenume,CNP)
VALUES ('100000','Popescu','Ioana','2750604123456')
```

Următorul update funcționează corect.

```
UPDATE vSalariatiiProductie
SET nume='Radulescu' WHERE cod=5
```

Fie vederea:

```
CREATE VIEW vAngajatiiProductie
```

```
AS SELECT * from tAngajati WHERE CodDepartament='Productie'
```

Dacă executăm:

```
UPDATE vAngajatiProductie  
SET CodDepartament='Administrativ'  
WHERE CodAngajat='5'
```

după modificare rândul corespunzător va dispărea din vedere (nu și din tabel) pentru că nu mai aparține departamentului 'Productie'.

Dacă se adaugă opțiunea WITH CHECK la definirea vederii atunci astfel de operații nu mai sunt permise – produc eroare.

```
CREATE VIEW vAngajatiProductie  
AS SELECT * from tAngajati WHERE CodDepartament='Productie'  
WITH CHECK OPTION
```

Modificarea vederilor

Modificarea unei vederi se poate face folosind comanda SQL alter view:

```
ALTER VIEW denumire_vedere  
[(alias [,alias]...)]  
[WITH ENCRYPTION]  
AS fraza_SELECT  
[WITH CHECK OPTION]
```

Ștergerea vederilor

Pentru ștergerea unei vederi se folosește comanda drop view:

```
drop view denumire_vedere;
```

Observație:

Deși vederile constituie un instrument extrem de convenabil pentru dezvoltatorul de aplicații, ele pot avea un impact negativ asupra performanței. Dacă pentru vederi simple (de exemplu simple copii ale unui tabel) impactul asupra performanței nu este sesizabil, pentru vederi complexe, care cuprind interogări pe mai multe tabele, acest impact poate fi foarte sever. De aceea folosirea vederilor trebuie planificată cu grija, pentru a se vedea dacă avantajul creat de simplitatea în manipulare a acestora compensează impactul negativ asupra performanței.

3. Desfășurarea lucrării

Probleme rezolvate:

1. Creați o vedere **EMPLOYEE** care să conțină codurile, numele, și departamentul angajaților din tabela EMP. Schimbați câpul din coloana EMPNO cu ANGAJAT. Vizualizați conținutul vederii.

Soluție:

```
CREATE OR REPLACE VIEW EMPLOYEE  
AS SELECT EMPNO, ENAME ANGAJAT, DEPTNO  
FROM EMP;
```

Pentru a vizualiza conținutul vederii se folosește `SELECT * FROM EMPLOYEE;`

Exemplu:

```
CREATE OR REPLACE VIEW employee  
AS SELECT empno,ename_angajat,deptno  
FROM emp1;
```

```
SQL> CREATE OR REPLACE VIEW EMPLOYEE  
2 AS SELECT EMPNO,ENAME_ANGAJAT,DEPTNO  
3 FROM EMP1;  
  
View created.
```

Rezultat:

```
SQL> SELECT* FROM EMPLOYEE;  
  
EMPNO  
-----  
ENAME_ANGAJAT                                DEPTNO  
-----  
FIRMA SRL  
MARIUS NICOLAE                                1  
  
FIRMA SRL2  
VASILE MARIN                                2  
  
FIRMA SRL3  
GRIGORE MIHAI                                3
```

2. Creați o vedere INFO_DEPT care să conțină numărul departamentului, salariul mediu (MEDIA), salariul maxim (MAXIM), salariul minim (MINIM), suma salariilor (SUMA), numărul de salarii (NO_SAL), numărul de comisioane (NO_COM) din tabela EMP.

Soluție:

```
CREATE OR REPLACE VIEW INFO_DEPT  
(DEPTNO, MEDIA, MAXIM, MINIM, SUMA, NO_SAL, NO_COM)  
AS SELECT DEPTNO, AVG(SAL), MAX(SAL), MIN(SAL), SUM(SAL),  
COUNT(SAL), COUNT(COMM)  
GROUP BY DEPTNO;
```

Exemplu:

```
SQL> ALTER TABLE EMP1  
2 ADD MEDIA NUMBER(5);  
  
Table altered.  
  
SQL> ALTER TABLE EMP1  
2 ADD MAXIM NUMBER(5);  
  
Table altered.  
  
SQL> ALTER TABLE EMP1  
2 ADD MINIM NUMBER(5);  
  
Table altered.  
  
SQL> ALTER TABLE EMP1  
2 ADD SUMA NUMBER(5);  
  
Table altered.  
  
SQL> ALTER TABLE EMP1  
2 ADD NO_SAL NUMBER(5);  
  
Table altered.  
  
SQL> ALTER TABLE EMP1  
2 ADD NO_COM NUMBER(5);  
  
Table altered.
```

Rezultat:

```
SQL> CREATE OR REPLACE VIEW INFO_DEPT
2  (DEPTNO,MEDIA,MAXIM,MINIM,SUMA,NO_SAL,NO_COM)
3  AS SELECT DEPTNO,AVG(NO_SAL),MAX(NO_SAL),MIN(NO_SAL),SUM(NO_SAL),COUNT(NO_SAL),COUNT(NO_COM)
4  FROM EMP1
5  GROUP BY DEPTNO;

View created.

SQL>
```

```
SQL> SELECT* FROM INFO_DEPT;
```

DEPTNO	MEDIA	MAXIM	MINIM	SUMA	NO_SAL	NO_COM
1	3	3	3	3	1	1
2	3	3	3	3	1	1
3	3	3	3	3	1	1

```
SQL>
```

3. Creați o vedere **DEPT20** conținând codul, numele și cod departament ale persoanelor din departamentul 20. Etichetați coloanele cu COD, PERSOANA, DEPTNO.

Nu permiți ca prin vedere o persoană să fie transferată într-un alt departament.

Soluție:

```
CREATE OR REPLACE VIEW DEPT20
(COD, PERSOANA, DEPTNO)
AS SELECT EMPNO, ENAME, DEPTNO
FROM EMP
WHERE DEPTNO=20
WITH CHECK OPTION CONSTRAINT depart20;

SELECT * FROM DEPT20;

UPDATE DEPT20
SET DEPTNO=10
WHERE EMPNO=7902;
```

4. Să se creeze vizualizarea v_emp_zod care să conțină codul și numele salariaților din tabelul emp_zod. Să se afișeze conținutul acesteia. Să se insereze o nouă înregistrare în această vizualizare. Ce observați? Să se șteargă vizualizarea v_emp_zod.

Soluție:

```
CREATE VIEW v_emp_zod (cod, nume)
AS SELECT employee_id, last_name
FROM emp_zod;

INSERT INTO v_emp_zod
VALUES (400,'N1');

Inserarea imi va da eroarea:
ORA-01400: cannot insert NULL into ("G233"."EMP_ZOD"."EMAIL")

DROP VIEW v_emp_zod;
```

5. Să se creeze vizualizarea v_emp_zod care să conțină codul, numele, emailul, data angajării, salariul și codul jobului salariaților din tabelul emp_zod. Să se insereze o nouă înregistrare în această vizualizare. Să se verifice că noua înregistrare a fost inserată și în tabelul de bază.

Soluție:

```
CREATE VIEW v_emp_zod
AS SELECT employee_id, last_name, email, hire_date,
salary, job_id FROM emp_zod;
INSERT INTO v_emp_zod
VALUES (400, 'N1', 'E1', SYSDATE, 5000, 'SA_REP');
SELECT employee_id, last_name, email, hire_date, salary,
job_id FROM emp_zod;
```

6. Să se mărească cu 1000 salariul angajatului având codul 400 din vizualizarea creată anterior. Ce efect va avea această acțiune asupra tabelului de bază?

Soluție:

```
UPDATE v_emp_zod
SET salary=salary+1000
WHERE employee_id = 400;
SELECT employee_id, last_name, salary
FROM emp_zod
WHERE employee_id = 400; // se modifica salariul si in tabelul de baza.
```

7. Să se șteargă angajatul având codul 400 din vizualizarea creată anterior. Ce efect va avea această acțiune asupra tabelului de bază?

Soluție:

```
DELETE FROM v_emp_zod
WHERE employee_id = 400;

SELECT employee_id, last_name, salary
FROM emp_zod
WHERE employee_id = 400; // angajatul este sters si din tabelul de baza.
```

8. Să se creeze vizualizarea v_emp_dept_zod care să conțină employee_id, last_name, hire_date, job_id, department_id din tabelul emp_zod și coloana department_name din tabelul dept_zod.

Soluție:

```
CREATE VIEW v_emp_dept_zod AS
SELECT employee_id, last_name, hire_date, job_id, e.department_id, department_name
FROM emp_zod e, dept_zod d
WHERE e.department_id = d.department_id; // este vizualizare complexa
```

9. Să se încerce inserarea înregistrării (500, 'N2', SYSDATE, 'SA_REP', 350, 'Administrativ') în vizualizarea creată anterior.

Soluție:

```
INSERT INTO v_emp_dept_zod
VALUES (500, 'N2', SYSDATE, 'SA_REP', 350, 'Administrativ');
```

Dă eroarea:

ORA-01779: cannot modify a column which maps to a non key-preserved table

10. Care dintre coloanele vizualizării v_emp_dept_zod sunt actualizabile?

Soluție:

```
SELECT    column_name, updatable
FROM      user_updatable_columns
WHERE     table_name = 'v_emp_dept_ZOD'; // nici o coloana nu este actualizabila
```

Probleme propuse spre rezolvare:

1. Să se creeze vizualizarea v_dept_zod care să conțină codul și numele departamentului, numărul de angajați din departamentul respectiv și suma alocată pentru plata salariilor. Această vizualizare permite actualizări?

2. Să se creeze vizualizarea v_manager_zod care să conțină numele, emailul, data angajării, salariul și codul jobului managerilor din tabelul emp_zod. În această vizualizare nu se va permite modificarea joburilor.

3. Să se listeze structura și conținutul vizualizării v_manager_zod. Să se încerce modificarea jobului unui angajat.

4. Să se creeze o vizualizare (v_dept_zod) asupra tabelului dept_zod să nu permită efectuarea nici unei operații LMD. Testați operațiile de inserare, modificare și ștergere asupra acestei vizualizări.

5. Să se creeze vizualizarea v_emp_info_zod asupra tabelului emp_zod care conține codul, numele, prenumele, emailul și numărul de telefon ale angajaților companiei. Se va impune unicitatea valorilor coloanei email și constrângerea de cheie primară pentru coloana corespunzătoare codului galeriei.

Observație: Constrângerile asupra vizualizărilor pot fi definite numai în modul *DISABLE NOVALIDATE*. Aceste cuvinte cheie trebuie specificate la declararea constrângerii, nefiind permisă precizarea altor stări.

6. Să se adauge o constrângere de cheie primară asupra vizualizării v_manager_zod.